

pharo-agentic-browser 概要

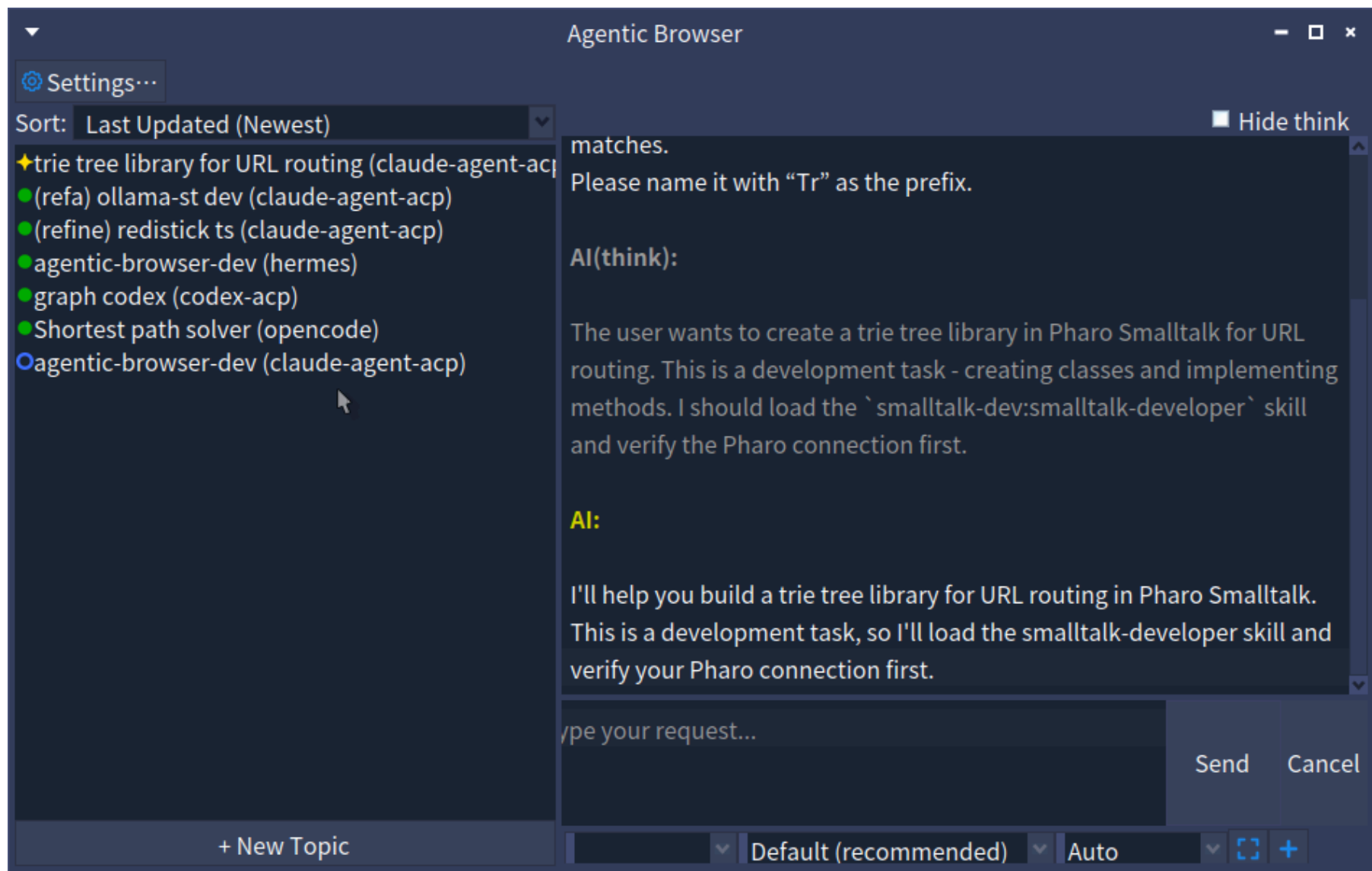
Pharoの統合AIコーディング環境

梅澤 真史

<https://github.com/mumez/pharo-agentic-browser>

pharo-agentic-browser とは？

Claude Code、Codex、OpenCode など、複数のAI コーディングエージェントを
Pharo から利用できる**統合AIコーディングツール**です。



各AIとのセッションは **トピック** として管理します:

1. トピックを作成し、ACP 対応エージェントを選択
2. リクエストを入力 + `@ClassName` でコードを参照、スクリーンショットの添付も可
3. AI が自律的に作業（タスク分解、コード変更、テスト）
4. AI が承認を必要とする場合、チャット内で一時停止して確認
5. プルダウンで応答すると AI が再開
6. トピックの状態は常にサイドバーで確認可能

開発の動機

AI コーディングエージェント専用の GUI ツールが標準になりつつあります:

- **Claude Desktop** — ツール、MCP、ファイルアクセスを備えた GUI版のClaude
- **Codex App** — OpenAI の自律的なコーディング環境
- **Cursor / Antigravity / Kiro** — AI ネイティブなエディタ

これらのツールは、AI エージェントとやり取りする際の敷居を下げ、単純なチャットを超えたものになっています。

変化の流れ: チャット → マルチエージェントへの委譲

時代	パラダイム	例
黎明期	エディタサイドバーでのチャット	ChatGPT, GitHub Copilot
少し前	エージェントモードを持つ AI ファーストな IDE	Cursor, Windsurf
現在	タスク全体をエージェントに委譲	Antigravity 2.0, Codex App

ツールは進化しています。UI はもはや「エディタ + チャット」ではなく、セッションのオーケストレーションへと向かっています。

なぜ Pharo にもこれが必要か

Pharo 開発者も同じパラダイムを享受すべき:

- リッチなライブ環境 — クラス、メソッド、ランタイムがすぐそこにある
- **pharo-acp** が複数エージェント向けの ACP クライアントをすでに提供
- 複数プロジェクトを1つのイメージ内で並行実行可能

複数の AI エージェントを制御できる Pharo ネイティブなツールが、次のステップとして自然です。

優位性	詳細
コンテキストを直接渡せる	<code>@ClassName</code> 、 <code>@Class>>method</code> 、スクリーンショット — コピー不要
エージェント非依存	pharo-acp 経由でさまざまなエージェントと連携
Pharo との統合	テスト、コードの変更監視がライブイメージの中で利用可能
並行セッション	複数エージェントを異なるトピックで同時実行

インストール

Pharo 12+ のイメージで Playground を開き、以下を評価:

```
Metacello new  
  baseline: 'AgenticBrowser';  
  repository: 'github://mumez/pharo-agentic-browser:main/src';  
  load: 'all'.
```

AgenticBrowserを開く:

```
AgenticBrowser open.
```

ACP 対応であれば、どのエージェントも利用可能:

エージェント	インストール
Claude Code	<code>npm install -g @agentclientprotocol/claude-agent-acp</code>
Codex	<code>npm install -g @agentclientprotocol/codex-acp</code>
Gemini CLI	ACP 組み込み (<code>gemini --acp</code>)
Copilot CLI	ACP 組み込み (<code>copilot --acp --stdio</code>)
Cursor CLI	ACP 組み込み (<code>agent acp</code>)

OpenCode、Kilo Code、Kiro CLI などにも対応

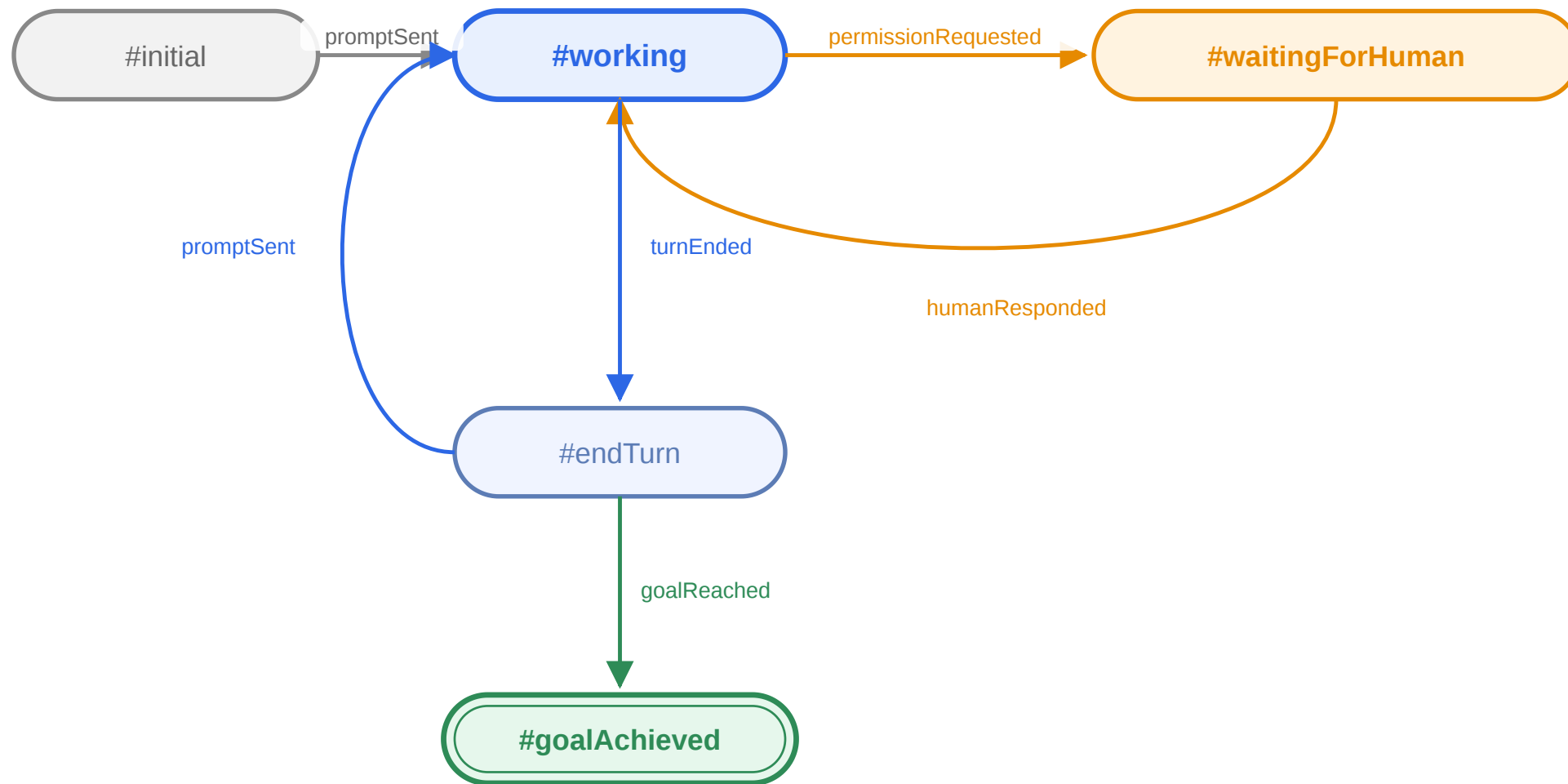
推奨: Smalltalk コードの質を上げるため、エージェントに [smalltalk-dev-plugin](#) をインストールしておく

基本機能

1. + **New Topic** をクリック
2. タイトルを入力し、コーディングエージェントを選択
3. (任意) 既存のプロジェクトディレクトリを設定
4. **Create** をクリック — トピックが左サイドバーに表示される
5. (任意) 右クリック → **Set Target Packages...** で追跡対象パッケージを設定

最初のメッセージには、Smalltalk開発者スキルを有効化するため、自動的に
`/st-buddy` が付与されます。

各トピックの状態はステートマシンで明確に管理:



アイコン	状態	意味
	working	AI が作業中
	waitingForHuman	AI が承認を待っている
	endTurn	ターン完了
	goalAchieved	ゴール達成

AI が承認を要求すると:

- ステータスが **?** (waitingForHuman) に変化
- **Send** ボタンが **Allow** に変化
- **Cancel** ボタンが **Deny** に変化

ボタンをクリックして応答すると、AI が再開します。

モーダルダイアログはありません。承認は会話フローの一部です。

チャット内で Pharo のクラスやメソッドを直接参照できます:

```
@QueryClass @DBAdapter>>connect please refactor this
```

AgenticBrowser は各 `@mention` を Tonel のソースとして解決し、ACP のテキストリソースにして添付します — コピペ不要。

ドラッグ&ドロップ

- System Browser から クラス をドラッグ → `@ClassName` を挿入
- System Browser から メソッド をドラッグ → `@ClassName>>methodName` を挿入

【 】 ボタンをクリックしてスクリーンショットを添付:

1. ボタンをクリック — カーソルが十字カーソルに変化
2. ドラッグして範囲を選択
3. @sc-20260528-001.png のようなメンションが入力欄に挿入される
4. 送信 — PNG が画像リソースとしてプロンプトに添付される

+ ボタンをクリックしてディスク上の任意のファイルを添付:

1. ボタンをクリック — ファイル選択ダイアログが開く
2. ファイルを選択 — `[filename]` のようなメンションが入力欄に挿入される
3. 送信 — ファイルの内容がテキストリソースとして添付される

サイズの大きいファイルは `maxAttachmentSize` (デフォルト 5MB) に切り詰められます。送信前にメンションテキストを削除すれば添付はキャンセルされます。

トピックを右クリック → **Set Goal...** で完了条件を入力:

```
all tests pass
```

AgenticBrowser は AI にゴール用のプロンプトを送信し、`result-<topic-id>.md` が作成されると、トピックは ☒ (`#goalAchieved`) に遷移します。

ゴール達成時にはアナウンサーやコールバックブロックのフック (whenGoalAchieved) も発火するので、独自の自動化ワークフローに統合できます。

トピックに関連するパッケージへの編集をイメージ内で監視:

- トピックを右クリック → **Set Target Packages...** で対象プレフィックスを設定
- 対象のクラス/メソッドが保存されると、確認の上でパッケージがエクスポートされる
- 追跡対象外の編集は候補として収集され、後から昇格可能

イメージ内でユーザ自身に変更した内容と、AI が見ている Tonel ソースとを同期させ続けます。

トピックは Pharo の **Fuel** シリアライザを使って `ab-topics.fuel` に自動保存され、イメージ再起動後も残ります。

手動での保存・復元:

```
AbTopicManager save.  
AbTopicManager load.
```

トピックごとの状態（設定、ステータス、会話）はすべて永続化されます。

カスタマイズ

新規トピックの作業ディレクトリは、`<agenticBrowserRoot>/topic-template` をもとに生成されます:

- デフォルトでは `smalltalk-dev-plugin` 向けに調整された `CLAUDE.md` / `AGENTS.md` を同梱
- `.claude`、`.opencode` などのエーเจント設定ディレクトリを配置できる — スキル、コマンド、ルールを全トピックで共有
- 独自にカスタマイズしたものに置き換え可能

新しいトピックごとにコーディングエーเจントを再設定する手間を省けます。

- MCP サーバー — AgenticBrowser のルートに `mcp.json` を配置。組み込みの `smalltalk-interop` / `smalltalk-validator` は自動マージ (`useDefaultMcpServers: false` で無効化可)
- カスタムエージェント — Playground または `ab-settings.json` からメニューにならないコーディングエージェントを登録

```
AbSettings default codingAgents: (AbSettings default codingAgents copyWith:
  {'name' -> 'my-agent'.
  'command' -> #('my-agent' '--acp')} asDictionary).
AbSettings save.
```

キー	デフォルト	説明
<code>useDefaultMcpServers</code>	<code>true</code>	組み込みの Smalltalk MCP サーバーをマージ
<code>aiPermissionWaitTimeoutSeconds</code>	<code>1800</code>	人間の承認待ちタイムアウト
<code>aiPermissionTimeoutOption</code>	<code>#reject_once</code>	自動応答: <code>allow_once</code> , <code>allow_always</code> , <code>reject_once</code>

設定は右クリック → **Edit Settings...** からトピックごとにも設定可能です。

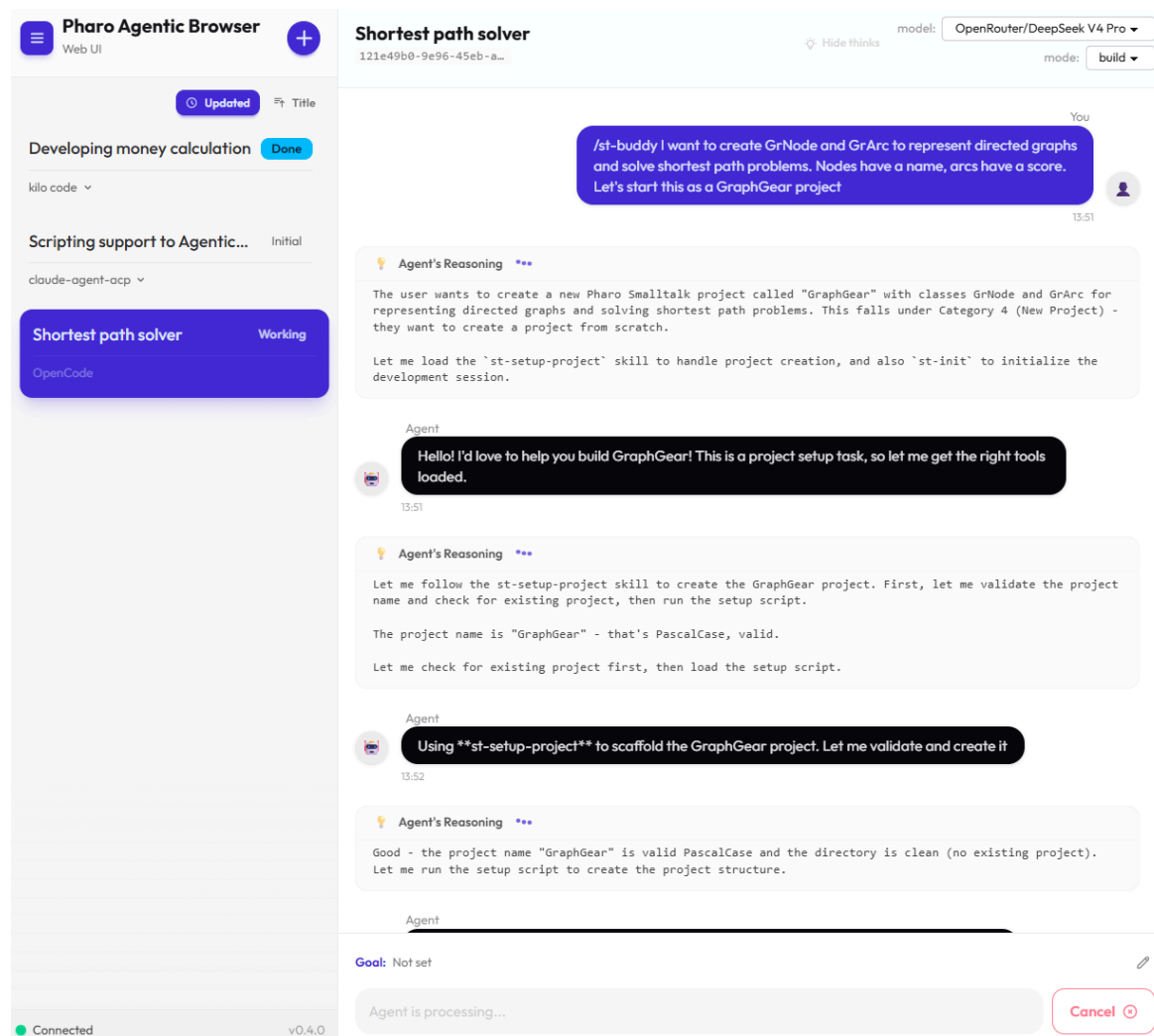
その他のインターフェース

任意の Web ブラウザから **AgenticBrowser** を利用できるオプションパッケージです。

- WebSocket フレームワーク [Ripple](#) により、リロード不要でリアルタイムに同期
- SolidJS + daisyUI でモバイル端末にも対応
- Spec UI にある操作（トピック管理、プロンプト送信、承認）はひととおり利用可能

ユースケース: 外出先からのスマートフォン確認、ディスプレイのないヘッドレス環境からのアクセスなど

→ 詳細は [Web UI スライド](#) を参照



UI操作なしにSmalltalkコードから **AgenticBrowser** のトピックをオーケストレーションできるオプションパッケージです。

- `seq:`、`para:`、`topicBy:`、`agentBy:` などわずか数個のメッセージでマルチエージェントのワークフローを構築
- 結果はステップ間で自動的に受け渡される — 手動での情報のやりとりは不要
- AI エージェント自身がスクリプトを書いて `st-eval` で実行することも可能 (`ab-scripting-feature-dev` スキル)

ユースケース: 定型的な AI ワークフローの実行や CI 組み込み、ヘッドレス環境での実行、複雑なマルチエージェント連携

→ 詳細は [Scripting API スライド](#) を参照

逐次ステップ (`seq:`) — 各トピックの結果は次のトピックのプロンプトに引き継がれる:

```
AgenticBrowser runBy: [ :builder |  
  builder seq: {  
    builder topicBy: [ :t |  
      t prompt: 'List 3 Pharo Smalltalk features in one sentence each.' ].  
    builder topicBy: [ :t |  
      t prompt: 'Summarize the feature list from the previous step in one sentence.' ]  
  } agentBy: [ :a | a claude ] ].
```

トピックは並行実行され、結果は次のステップのためにまとめられる:

```
AgenticBrowser runBy: [ :builder |  
  builder para: {  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk language features.' ] .  
    builder topicBy: [ :t | t prompt: 'List 3 Pharo Smalltalk development tools.' ]  
  } agentBy: [ :a | a claude ] ].
```

`seq:` と `para:` は自由に組み合わせ可能

— 例: 並列調査して結果をまとめる → 結果をもとに逐次実行

実践例: *To-Do アプリ*

[to-do-list-orchestration-script.md](#)

まとめ

pharo-agentic-browser は、コーディングエージェントへの委譲というパラダイムを Pharo にもたらしめます:

- **ネイティブ GUI** — イメージから離れずに複数の AI セッションを管理
- **エージェント非依存** — ACP 対応エージェントであればどれでも利用可能
- **リッチなコンテキスト** — コードメンション、ドラッグ&ドロップ、スクリーンキャプチャ
- **Human-in-the-Loop** — 会話の中で承認、割り込みダイアログなし
- **ゴール駆動 & 拡張可能** — 完了条件やフック指定、MCP サーバー/エージェントのカスタマイズに対応
- **Web UI / Scripting API** — ブラウザからの利用や、コード駆動のオーケストレーションにも対応

フィードバックとコントリビューションをお待ちしています！

<https://github.com/mumez/pharo-agentic-browser>